

Cloud Security, Cloud Security Posture Management & Single Sign-On

*A comprehensive, real-world course connecting
core cloud security concepts to the systems behind
Amazon, Flipkart, Swiggy, Zomato, Lenskart, and
modern banking platforms*

Author: Dinesh Wadhvani

Course Material & Instructor Script | Level: Introductory to Intermediate
Prepared for classroom delivery with diagrams, case studies & exercises

Table of Contents

Course Overview & How to Use This Material

Module 1 — Introduction to Cloud Security (Lecture 1, Hour 1)

- 1.1 Cloud Computing Recap: Service & Deployment Models
- 1.2 Why Cloud Security Is a Different Discipline
- 1.3 The Shared Responsibility Model
- 1.4 Case Study: The Capital One Breach (2019)
- 1.5 The CIA Triad in the Cloud
- 1.6 The Five Pillars of Cloud Security
- 1.7 The Cloud Security Threat Landscape
- 1.8 Case Study: Architecting Security for a Food-Delivery Platform
- 1.9 Classroom Exercise 1 — Whose Responsibility Is It?
- 1.10 Module 1 Summary & Key Takeaways

Module 2 — Cloud Security Posture Management (CSPM) (Lecture 1, Hour 2)

- 2.1 Why CSPM Exists: The Misconfiguration Problem
- 2.2 What Is CSPM? A Formal Definition
- 2.3 How a CSPM Platform Works
- 2.4 The Continuous Cloud Security Cycle
- 2.5 Compliance Frameworks CSPM Maps To
- 2.6 The Cloud-Native Security Landscape: CSPM, CWPP, CIEM, KSPM, CNAPP
- 2.7 Popular CSPM Tools in the Industry
- 2.8 Case Study: Anatomy of an Exposed Storage Bucket
- 2.9 Case Study: Multi-Cloud Posture at Hyper-Growth Scale
- 2.10 Classroom Exercise 2 — Spot the Misconfiguration
- 2.11 Module 2 Summary & Key Takeaways

Module 3 — Single Sign-On (SSO) Authentication (Lecture 2, Hours 3–4)

- 3.1 The Problem: Password Fatigue
- 3.2 What Is Single Sign-On?
- 3.3 Core Actors: User, Identity Provider, Service Provider
- 3.4 Authentication vs. Authorization
- 3.5 SAML 2.0 — Enterprise SSO
- 3.6 OAuth 2.0 — Delegated Authorization
- 3.7 OpenID Connect & the JSON Web Token
- 3.8 SAML vs. OAuth vs. OIDC — Choosing the Right Protocol
- 3.9 Case Study: SSO in Banking & Net-Banking Ecosystems
- 3.10 Case Study: Lenskart — Social Login & OTP-Based SSO
- 3.11 SSO Security Risks & Mitigations
- 3.12 SSO + MFA: Stronger Together
- 3.13 Enterprise SSO Tools Landscape
- 3.14 Classroom Exercise 3 — Map the OAuth Login Flow
- 3.15 Classroom Exercise 4 — Rapid-Fire Protocol Quiz
- 3.16 Module 3 Summary & Key Takeaways

Course Wrap-Up, Glossary & References

- [Recap & Interview / Exam Pointers](#)
- [Glossary of Key Terms](#)
- [Answer Key for Classroom Exercises](#)
- [Further Reading & References](#)

Course Overview & How to Use This Material

This course material accompanies a two-lecture, four-hour classroom module on cloud security, delivered as part of the Symbiosis guest lecture / FDP series. It is written to be read in two ways: as **pre-reading or revision material for students**, and as a **content reference for the instructor** while preparing slides and in-class discussion.

The material deliberately anchors every abstract concept to a system students already use every day — ordering food on **Swiggy** **Zomato**, shopping on **Amazon** **Flipkart**, buying eyewear on **Lenskart**, or logging into a bank's net-banking portal. The intent is that a student who finishes this course can explain, in concrete terms, what happens the instant they tap "Login with Google" on their phone — and why the security team behind that app cares about far more than just the login screen.

LEARNING OUTCOMES

By the end of this course, learners will be able to:

- Explain the Shared Responsibility Model and correctly assign a given security control to "customer" or "cloud provider" for IaaS, PaaS, and SaaS.
- Describe why cloud misconfigurations are the leading cause of cloud data breaches, and how Cloud Security Posture Management (CSPM) tools continuously detect and help remediate them.
- Distinguish CSPM from adjacent disciplines (CWPP, CIEM, KSPM, CNAPP).
- Explain Single Sign-On (SSO) conceptually, and trace, step by step, how SAML, OAuth 2.0, and OpenID Connect actually move a user from "not logged in" to "authenticated" across real consumer and enterprise apps.
- Identify common SSO-related security risks and the mitigations used in production systems.

Session	Duration	Topics Covered
Lecture 1	2 hours	Module 1: Introduction to Cloud Security (60 min) + Module 2: Cloud Security Posture Management, CSPM (60 min)
Lecture 2	2 hours	Module 3: Single Sign-On, SSO (120 min, including protocol deep-dives, two case studies, and two classroom exercises)

Throughout the material you will see four recurring boxes: a blue **Example** box that connects a concept to a real platform, a green **Key Point** box that distills the one idea worth remembering, an amber **Caution** box that flags a common mistake or real-world risk, and a purple **Classroom Exercise** box with a ready-to-run activity, discussion prompt, or short quiz.

Introduction to Cloud Security

1.1 Cloud Computing Recap: Service & Deployment Models

Before discussing cloud *security*, it is worth a sixty-second recap of what "the cloud" actually is, because the shared responsibility model in the next section only makes sense against this backdrop. Cloud computing is the on-demand delivery of computing resources — servers, storage, databases, networking, software — over the internet, billed on a pay-as-you-go basis instead of being bought and run in a private data center.

Cloud services are usually grouped into three service models:

- **IaaS (Infrastructure as a Service)** — the provider gives you raw virtual machines, storage, and networking; you install and manage everything above that. Example: Amazon EC2, Azure Virtual Machines.
- **PaaS (Platform as a Service)** — the provider also manages the operating system and runtime; you deploy application code. Example: AWS Elastic Beanstalk, Azure App Service, Google App Engine.
- **SaaS (Software as a Service)** — the provider runs the entire application; you simply use it through a browser or app. Example: Salesforce, Zoom, Microsoft 365, Workday.

Cloud services are also deployed in different models — **public cloud** (shared infrastructure, e.g. AWS, Azure, GCP), **private cloud** (dedicated to one organization, common in large banks handling regulated data), and **hybrid cloud** (a mix of both, which is how most large Indian enterprises — including many banks and NBFCs — actually operate: sensitive core-banking data stays on-premises or in a private cloud, while customer-facing apps and analytics run on public cloud).

EXAMPLE — EVERYDAY CLOUD USAGE

When you order food on **Swiggy**, the app itself is likely running on IaaS/PaaS (virtual machines or managed container services), it stores your order history in a managed database (PaaS), and it may use SaaS tools internally for support ticketing or analytics dashboards. A single company routinely uses all three service models at once.

1.2 Why Cloud Security Is a Different Discipline

Traditional, on-premises security assumed a fairly fixed perimeter: your servers sat in your own data center, behind your own firewall, and "inside" usually meant "trusted." Cloud computing dissolves that perimeter. Your application's database might sit in a data center you have never seen, your traffic might cross the public internet between microservices, and a single mislicked checkbox in a web console can expose data to the entire internet in seconds. Three shifts make cloud security a distinct discipline rather than "the same security, somewhere else":

- **Shared control.** You no longer control every layer of the stack — some of it is the provider's job, some is yours, and the split changes depending on whether you use IaaS, PaaS, or SaaS.
- **Speed and scale.** Cloud infrastructure is created and destroyed through code (Infrastructure as Code) in seconds. A single engineer can accidentally provision thousands of insecure resources before anyone notices — something physically impossible in a hardware data center.
- **API-driven everything.** Every cloud resource is reachable and configurable through an API. That is enormously powerful — and it means a leaked API key or an over-permissioned identity can do as much damage as a stolen physical server key.

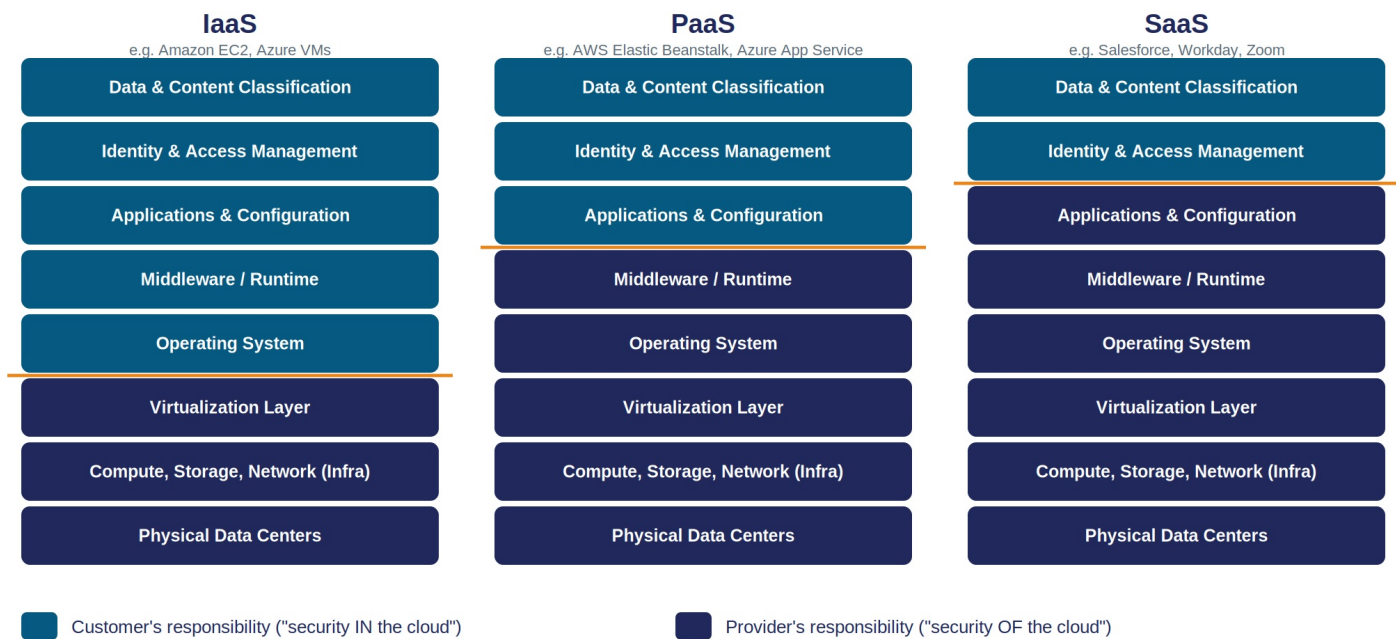
1.3 The Shared Responsibility Model

The single most important mental model in cloud security is the **Shared Responsibility Model**. Every major cloud provider (AWS, Microsoft Azure, Google Cloud) publishes a version of it, and every cloud security incident can usually be traced back to someone misunderstanding where the line sits.

The provider is always responsible for the **security OF the cloud** — the physical data centers, the host hardware, the virtualization layer, and (depending on the service) the underlying network. The customer is always responsible for **security IN the cloud** — how they configure identity and access, how they classify and protect their data, how they configure their applications and networks. What changes across IaaS, PaaS, and SaaS is exactly *where* that line sits, as shown below.

The Cloud Shared Responsibility Model

Who secures what changes as you move from IaaS to SaaS



Real-world lesson: In the 2019 Capital One breach, AWS secured the underlying infrastructure correctly — but a misconfigured Web Application Firewall on the customer side exposed 100M+ records. That is the customer's side of the line.

Figure 1.1 — The Cloud Shared Responsibility Model across IaaS, PaaS, and SaaS.

Notice the pattern: as you move from IaaS to SaaS, the provider takes on more of the stack, but your responsibility for **data** and **identity & access management** never goes away. This is the single most commonly misunderstood point by students and, frankly, by many working engineers: "we're on SaaS, so security is the vendor's problem" is a sentence that has caused real breaches.

EXAMPLE — SAME PROVIDER, DIFFERENT RESPONSIBILITY

On **Amazon Web Services**, if you run your own application on an EC2 virtual machine (IaaS), *you* are responsible for patching the operating system. If you instead use AWS Lambda (a serverless / PaaS-like service) to run the same code, AWS patches the underlying OS for you — but you are still 100% responsible for what your code does with the data it processes, and for who is allowed to invoke that function.

1.4 Case Study: The Capital One Breach (2019)

CASE STUDY

In 2019, Capital One — a major US bank running its infrastructure on AWS — suffered a breach affecting over 100 million customer records, including credit scores, balances, and Social Security numbers.

The root cause was **not** a flaw in AWS's infrastructure. A former employee of a third-party firm exploited a **misconfigured Web Application Firewall (WAF)** that Capital One's own team had set up. The misconfigured WAF had excessive permissions and could be tricked into requesting temporary security credentials from the underlying instance metadata service, which were then used to list and read data from Capital One's own S3 storage buckets.

AWS secured "the cloud" exactly as promised. The customer-side misconfiguration — on the customer's side of the shared responsibility line — is what caused the breach. This is the canonical teaching example for why the shared responsibility model matters, and it is exactly the class of problem that CSPM (Module 2) is designed to catch automatically, continuously, before an attacker finds it.

1.5 The CIA Triad in the Cloud

The classic **CIA triad** — Confidentiality, Integrity, Availability — still anchors cloud security, but each pillar takes on a cloud-specific flavor:

Pillar	Traditional Meaning	Cloud-Specific Concern
--------	---------------------	------------------------

Confidentiality	Only authorized people see data	Public storage buckets, over-shared links, cross-tenant data leakage in multi-tenant SaaS
Integrity	Data isn't tampered with	Unauthorized changes to Infrastructure-as-Code, unsigned API calls, supply-chain tampering of container images
Availability	Systems stay up	DDoS against public endpoints, accidental resource deletion via automation, regional cloud outages

1.6 The Five Pillars of Cloud Security

In practice, a cloud security program is built around five interlocking pillars. This course focuses deeply on the first and last of these (identity, via SSO, and posture, via CSPM), but a professional should be conversationally fluent in all five:

1. **Identity & Access Management (IAM)** — who can do what, enforced through the principle of least privilege (grant only the permissions actually required, nothing more). Amazon's own IAM service lets a retail platform grant its "order processing" microservice permission to read the orders table — and explicitly deny it permission to touch the payments table.
2. **Data Protection** — encryption at rest (data sitting in storage) and in transit (data moving across the network), tokenization of sensitive fields such as card numbers, and data classification so teams know what is sensitive in the first place.
3. **Network Security** — Virtual Private Clouds (VPCs), security groups and firewall rules, segmentation so that a compromised front-end service cannot directly reach a payments database.
4. **Compliance & Governance** — mapping technical controls to frameworks and regulations such as ISO 27001, PCI-DSS (payment card data), and, in India, the RBI's IT framework for banks and the Digital Personal Data Protection (DPDP) Act.
5. **Monitoring, Detection & Response** — logging, alerting, and the continuous posture-checking that Module 2 covers in depth.

1.7 The Cloud Security Threat Landscape

With the model and the pillars in place, it helps to name the actual threats a cloud security program defends against. The six categories below recur, in one form or another, across almost every publicly documented cloud breach of the last decade.



skill to exploit, which is exactly why it is so common and so damaging.

- **Account/credential hijacking** increasingly targets session tokens rather than passwords — which is directly relevant to Module 3, since a stolen SSO session token can be just as dangerous as a stolen password, sometimes more so, because it may bypass the login and MFA step entirely.

1.8 Case Study: Architecting Security for a Food-Delivery Platform

To make these ideas concrete, consider a simplified architecture that mirrors how a platform such as **Swiggy**, **Zomato**, or an e-commerce app like **Flipkart** is actually built: a mobile/web front-end, a login layer, an API gateway, a set of backend microservices running inside a private network, and a data layer.

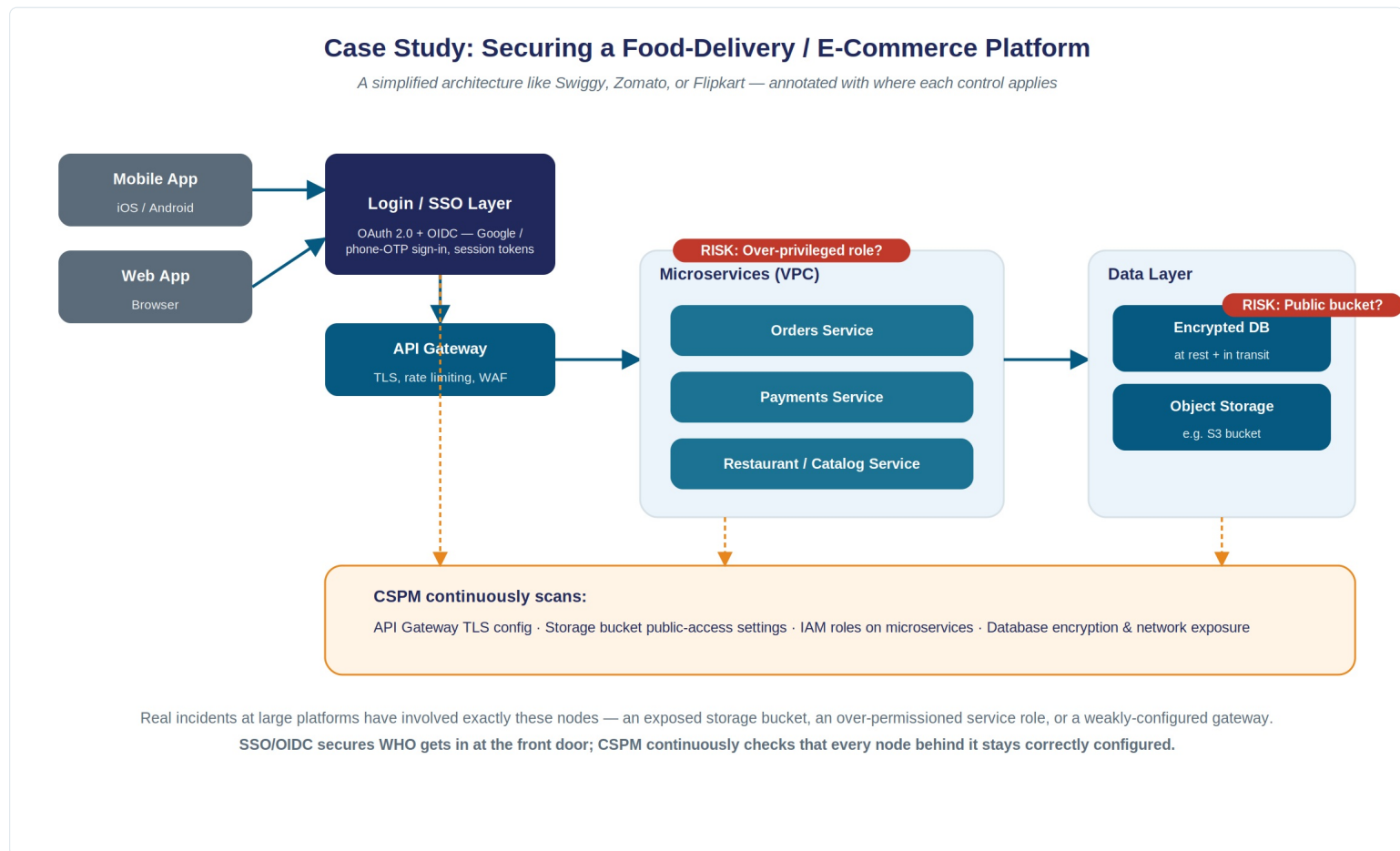


Figure 1.3 — Where security controls apply across a typical consumer-app architecture. Note the login/SSO layer at the front door and the continuous CSPM scanning shown underneath — the two halves of this course, working together.

KEY POINT

Notice that this single diagram already contains both halves of this course: the **Login / SSO layer** (Module 3) decides *who* is allowed in, while **CSPM** (Module 2) continuously checks that every other node in the system — the gateway, the microservices, the storage, the database — stays correctly configured. Identity and posture are the two questions a mature cloud security program must always be able to answer: *who is this, and is everything behind them still locked down?*

1.9 Classroom Exercise 1 — Whose Responsibility Is It?

CLASSROOM EXERCISE · 10-12 MINUTES

Split the class into pairs. For each scenario below, the pair must decide: is this the **Cloud Provider's** responsibility, the **Customer's** responsibility, or **Shared**? Ask a few pairs to justify their answer out loud before revealing the suggested answer (Appendix, Answer Key).

1. Patching a vulnerability in the physical server's firmware.
2. Setting an S3 / cloud storage bucket to "private" instead of "public."
3. Enforcing multi-factor authentication for admin console logins.
4. Encrypting customer data before storing it in a managed database.
5. Ensuring the data-center building has backup power and physical guards.
6. Configuring which microservices are allowed to talk to which others inside a Virtual Private Cloud.
7. Patching the hypervisor that runs customer virtual machines.

1.10 Module 1 Summary & Key Takeaways

KEY TAKEAWAYS

- Cloud security is a **shared** responsibility — the provider secures the cloud, the customer secures what they put in it, and the split moves as you go from IaaS to SaaS.
- Data protection and identity & access management remain the customer's job in every single service model, with no exceptions.
- Real breaches (Capital One being the textbook example) are usually failures of configuration and process, not exotic cryptographic attacks.
- A cloud security program rests on five pillars: IAM, data protection, network security, compliance & governance, and monitoring & response.
- The six recurring threat categories — misconfiguration, weak identity, insecure APIs, data exposure, account hijacking, and availability attacks — will reappear, by name, throughout Modules 2 and 3.

Cloud Security Posture Management (CSPM)

2.1 Why CSPM Exists: The Misconfiguration Problem

Module 1 closed with a simple but important observation: misconfiguration, not sophisticated hacking, is the leading cause of cloud data breaches. Industry breach reports year after year identify misconfigured cloud resources — open storage, overly permissive firewall rules, disabled logging, unencrypted databases exposed to the internet — as the single largest category of root cause.

Three forces make this problem worse at scale, and explain why manual review can never keep up:

- **Volume.** A mid-size company can easily run tens of thousands of cloud resources across multiple accounts, regions, and providers.
- **Velocity.** Infrastructure as Code and CI/CD pipelines mean new resources are created and changed continuously, all day, by many different engineers.
- **Drift.** A resource that was configured correctly on Monday can be manually "fixed" by an engineer under pressure on Friday in a way that quietly reopens a security gap — with nobody re-reviewing it.

This is precisely the gap that **Cloud Security Posture Management (CSPM)** was built to close.

2.2 What Is CSPM? A Formal Definition

DEFINITION

Cloud Security Posture Management (CSPM) is a category of security tooling that continuously and automatically discovers cloud resources, evaluates their configuration against security best practices and compliance frameworks, and surfaces — or automatically fixes — any misconfiguration or policy violation it finds, across every cloud account an organization owns.

Three words in that definition matter more than the rest: **continuously** (not a one-time audit), **automatically** (not a human clicking through consoles), and **every account** (most real organizations are multi-cloud or at least multi-account, and a gap in visibility is itself a security gap).

2.3 How a CSPM Platform Works

Functionally, every CSPM platform — whether it is a cloud-native service or a third-party product — follows the same architecture: it connects to your cloud accounts using read-only API credentials, builds a live inventory of every resource, evaluates that inventory against a policy library, and pushes prioritized findings out to the teams and tools that need to act on them.

How a CSPM Platform Works

From raw cloud accounts to a prioritized, remediated risk list

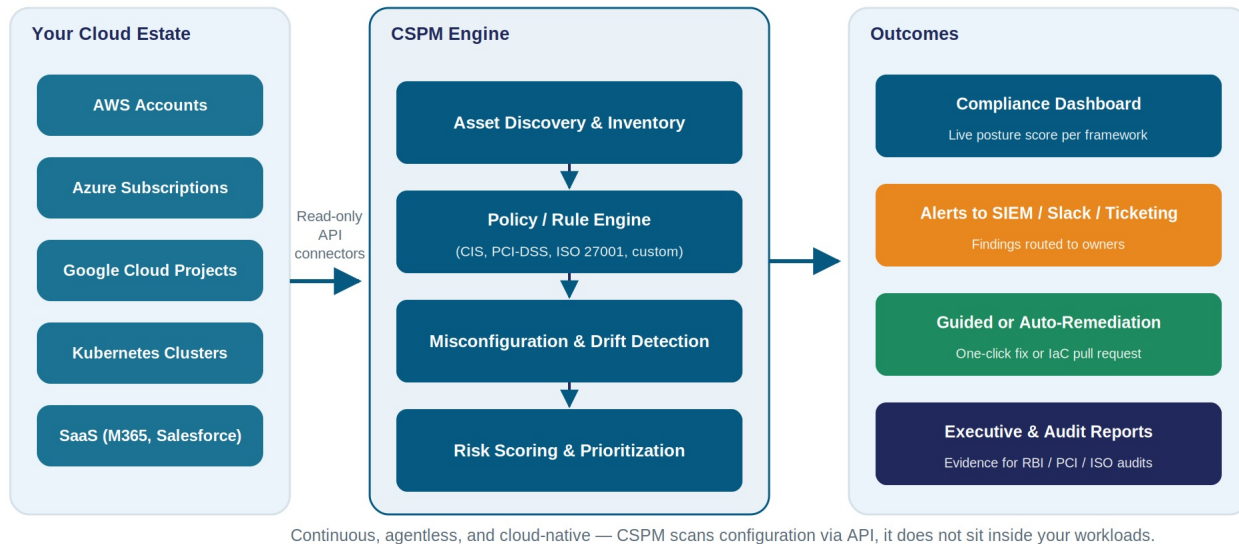


Figure 2.1 — How a CSPM platform connects cloud accounts to outcomes: dashboards, alerts, remediation, and audit reports.

Notice that CSPM is **agentless** — it does not install software inside your virtual machines or containers. It reads configuration through the cloud provider's own management APIs, the same APIs a console user or an Infrastructure-as-Code tool would use. This is an important distinction from CWPP (Cloud Workload Protection, covered in section 2.6), which does run inside the workload.

2.4 The Continuous Cloud Security Cycle

CSPM is not a report you generate once a quarter for an audit. It is a closed loop that runs continuously:

The Continuous Cloud Security Cycle

CSPM is not a one-time scan — it is a loop that never stops

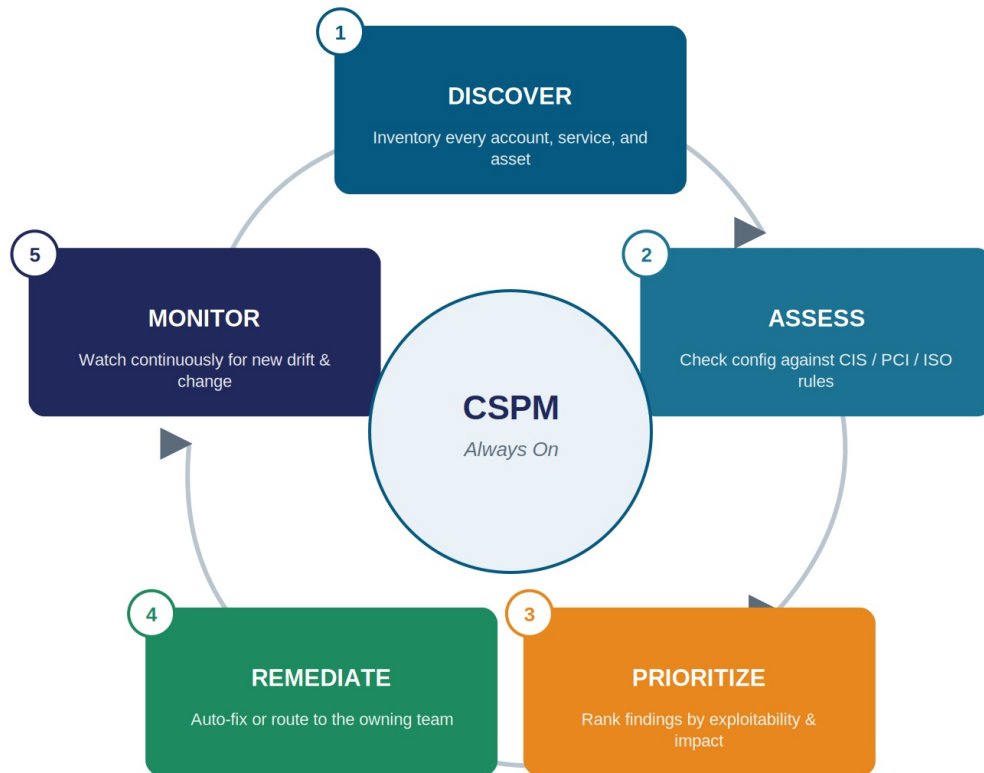


Figure 2.2 — The five-stage continuous cycle every CSPM platform runs, endlessly, in the background.

1. **Discover.** Build and maintain a live inventory of every account, service, and asset — including "shadow" resources an individual engineer spun up without telling anyone.
2. **Assess.** Check every discovered resource's configuration against rules derived from frameworks like the CIS Benchmarks, PCI-DSS, and ISO 27001, plus any custom organizational rules.
3. **Prioritize.** Not every finding is equally urgent. A public storage bucket containing customer PII is a five-alarm fire; an unused test resource with a minor misconfiguration is not. Good CSPM tools rank findings by exploitability and business impact, not just by raw count.
4. **Remediate.** Fix the issue — either automatically (for well-understood, low-risk fixes) or by routing a clear, actionable ticket to the owning engineering team, ideally with a one-click fix or a ready-made Infrastructure-as-Code pull request.
5. **Monitor.** Watch continuously for new resources and new configuration drift, and start the loop again — forever.

2.5 Compliance Frameworks CSPM Maps To

A major reason organizations adopt CSPM is not only "better security" in the abstract, but **demonstrable, continuous compliance**. CSPM platforms ship with pre-built rule packs mapped to the frameworks auditors actually check against:

Framework	Relevance
CIS Benchmarks	Vendor-neutral, highly detailed hardening guidelines for AWS, Azure, GCP — the most common baseline CSPM tools ship out of the box.
PCI-DSS	Mandatory for any platform that stores, processes, or transmits payment card data — directly relevant to e-commerce and food- delivery apps that handle card payments.
ISO/IEC 27001	The international standard for information security management systems; widely required in enterprise vendor contracts.
RBI IT Framework / Cyber Security Framework	Reserve Bank of India requirements for banks and NBFCs, including strict controls on data localization, encryption, and access governance — central to the banking case study in Module 3.
DPDP Act, 2023	India's Digital Personal Data Protection Act, which raises the bar on how personal data of Indian users must be

2.6 The Cloud-Native Security Landscape: CSPM, CWPP, CIEM, KSPM, CNAPP

CSPM is one member of a family of closely related, frequently confused acronyms. Understanding how they relate is a common interview and exam question in its own right.

Where CSPM Fits: The Cloud-Native Security Landscape

Four acronyms, one goal — secure the full cloud-native stack

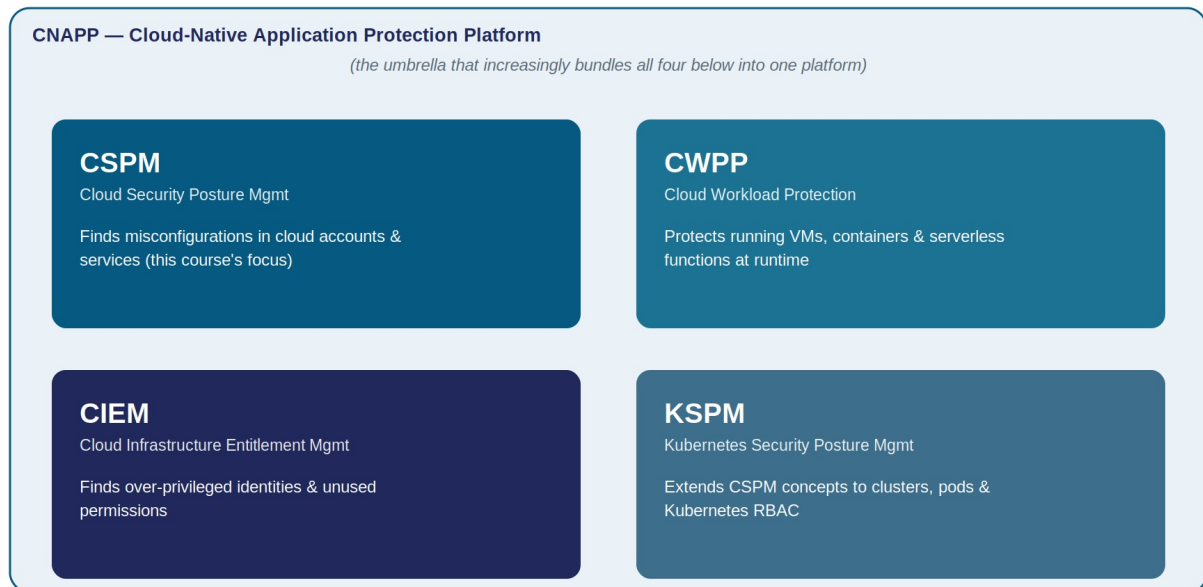


Figure 2.3 — CSPM sits inside a broader family of cloud-native security disciplines, increasingly bundled under the CNAPP umbrella.

In one sentence each: **CSPM** asks "is this cloud resource configured securely?" **CWPP** asks "is this running workload (VM, container, function) behaving securely right now?" **CIEM** asks "does this identity have more access than it actually needs?" and **KSPM** asks the CSPM question specifically for Kubernetes clusters, pods, and role-based access control. **CNAPP** is the umbrella term for platforms that bundle all of the above into a single product — the direction the CSPM market has been moving for several years.

2.7 Popular CSPM Tools in the Industry

Category	Tools	Notes
Cloud-native	AWS Security Hub & AWS Config, Microsoft Defender for Cloud, Google Security Command Center	Built into the provider; strongest within that one provider's ecosystem, weaker across multi-cloud.
Third-party / independent	Wiz, Palo Alto Prisma Cloud, Orca Security, Microsoft Defender CSPM (multi-cloud), CrowdStrike Falcon Cloud Security	Built to be multi-cloud from day one; often add CWPP, CIEM, and vulnerability scanning in one platform (i.e. a CNAPP).

EXAMPLE — MULTI-CLOUD REALITY

A fast-growing company like **Lenskart** or **Flipkart**, operating at large scale with many engineering teams, will very often end up running workloads across more than one cloud provider, plus a large number of SaaS tools for internal operations. A third-party CSPM tool that gives one unified view across all of them is frequently more valuable than any single provider's native tool precisely because it removes the blind spots between providers.

2.8 Case Study: Anatomy of an Exposed Storage Bucket

CASE STUDY — A COMPOSITE, REALISTIC SCENARIO

Consider a large e-commerce platform's marketing team that needs to share a folder of product images with an external agency. An engineer creates a cloud storage bucket, and — to avoid back-and-forth over access requests — sets its access policy to "public read" so the agency can simply access the files by URL. The bucket is never revisited.

Eighteen months later, that same bucket is quietly being reused by another internal team to stage a data export for a nightly analytics job — a CSV containing partial customer order data, dropped into the same bucket "just for a few hours" before being deleted. The public read policy is still set. Search-engine crawlers and automated internet scanners — tools that continuously scan the entire public cloud IP space for exactly this kind of exposure — find the file within hours.

This exact pattern — a bucket opened for a legitimate, narrow reason, then never re-locked down, then reused for something more sensitive — mirrors a large share of publicly reported cloud storage exposures across industries, including at well-known e-commerce and consumer platforms. A CSPM tool would have flagged the "public read" policy the moment it was set, and again every single day it remained public, regardless of what was later placed inside it.

2.9 Case Study: Multi-Cloud Posture at Hyper-Growth Scale

Consider a consumer platform in hyper-growth mode — the kind of trajectory companies like **Lenskart** or a modern food-delivery unicorn go through. Engineering teams multiply quickly, each often given autonomy to ship fast. Backend services increasingly move to containers orchestrated with Kubernetes, spread across multiple cloud regions and sometimes multiple providers for redundancy.

In this environment, CSPM's job extends naturally into **KSPM**: checking that Kubernetes cluster API servers are not exposed to the public internet, that pod security policies prevent containers from running with excessive privileges, and that role-based access control (RBAC) inside the cluster follows least-privilege — the same underlying philosophy as classic CSPM, applied one layer deeper into the stack.

2.10 Classroom Exercise 2 — Spot the Misconfiguration

CLASSROOM EXERCISE · 12-15 MINUTES

Display or hand out the following simplified cloud storage bucket policy (plain-English paraphrase of a real cloud IAM policy document). In small groups, students identify what is wrong and how they would fix it. Discuss as a class, then reveal the suggested answer.

Setting	Configured Value
Bucket name	prod-customer-exports
Public access	Allowed (read + list)
Encryption at rest	Disabled
Access logging	Disabled
Versioning	Disabled
Attached IAM role	"AdministratorAccess" (full account access)

Guiding questions: Which single setting is the most urgent to fix, and why? Which setting, if fixed, would have made the *impact* of the breach smaller even if the bucket had still been found? Which setting is a violation of least privilege rather than a data- exposure issue?

2.11 Module 2 Summary & Key Takeaways

KEY TAKEAWAYS

- CSPM continuously discovers, assesses, prioritizes, and helps remediate cloud misconfigurations — it is a loop, not a one-time audit.
- CSPM is agentless: it reads configuration through cloud provider APIs, unlike CWPP, which protects running workloads from the inside.
- CSPM, CWPP, CIEM, and KSPM are increasingly bundled together as CNAPP — a Cloud-Native Application Protection Platform.
- CSPM findings map directly to compliance frameworks (CIS, PCI-DSS, ISO 27001, RBI, DPDP Act) — making CSPM as much a compliance tool as a security tool.
- The "exposed bucket" pattern — a resource opened for a narrow, legitimate reason and never re-locked down — is the single most common real-world misconfiguration story, and exactly what CSPM is built to catch.

BRIDGE TO LECTURE 2

Module 1 and Module 2 answered "is everything behind the front door configured securely?" Lecture 2 turns to the front door itself: *who* is allowed in, and how a modern application actually verifies that, without asking the user to manage yet another password. That is Single Sign-On.

Single Sign-On (SSO) Authentication

3.1 The Problem: Password Fatigue

Think about how many apps a typical smartphone owner in India uses in a single week: **Swiggy** or **Zomato** for food, **Flipkart** or **Amazon** for shopping, **Lenskart** for eyewear, a net-banking app, a UPI app, a workplace email account, and half a dozen more. If every one of these required a separate, unique, strong password, one of two things happens in practice: people reuse the same password everywhere (so one leaked password compromises many accounts), or people simply give up and use the "forgot password" flow so often that it becomes a support burden. Security teams call this **password fatigue**, and it is the human problem that Single Sign-On exists to solve.

3.2 What Is Single Sign-On?

DEFINITION

Single Sign-On (SSO) is an authentication scheme that allows a user to log in once, with one set of credentials, and gain access to multiple independent applications or services — without having to log in again separately to each one.

Single Sign-On: One Login, Many Applications

Without SSO, a user re-authenticates — and re-uses passwords — everywhere

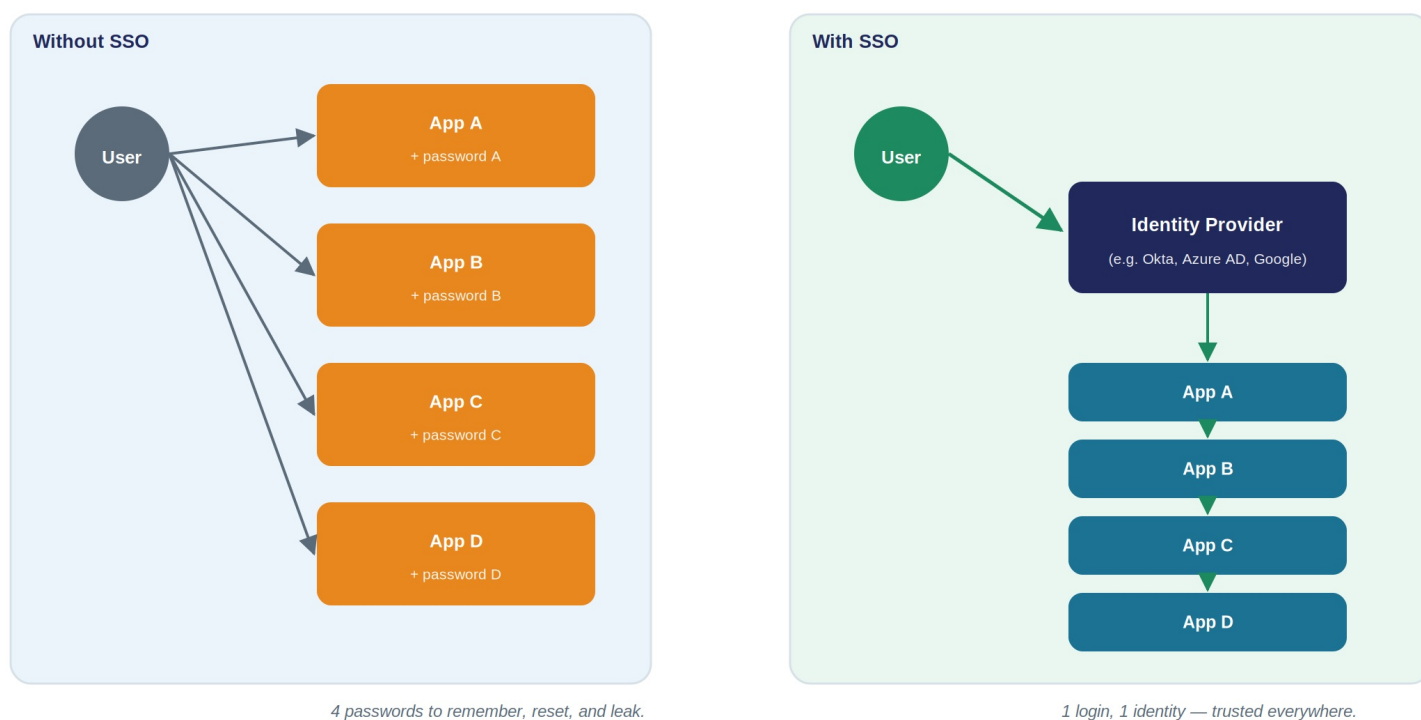


Figure 3.1 — Without SSO, a user manages one password per app. With SSO, one trusted Identity Provider vouches for the user everywhere.

You already use SSO constantly. Every time you tap "Continue with Google" or "Continue with Facebook" on a new app, or every time your office laptop logs you into email, chat, and your HR portal without asking for a password each time, you are using SSO.

3.3 Core Actors: User, Identity Provider, Service Provider

Every SSO conversation involves three roles:

- **The User** — the human trying to log in.
- **The Identity Provider (IdP)** — the trusted system that actually verifies who the user is (checks the password, sends the OTP, enforces MFA) and issues a signed proof of identity. Examples: Google, Microsoft Entra ID (formerly Azure AD), Okta, a bank's own authentication server.
- **The Service Provider (SP)**, sometimes called the **Relying Party** — the application the user actually wants to use, which trusts the IdP's word instead of checking the password itself. Examples: Workday, Salesforce, or a food-delivery app that

accepts "Sign in with Google."

The entire security value of SSO rests on one design decision: the Service Provider **never sees or handles the user's actual password**. It only ever receives a signed statement from the IdP saying, in effect, "I have verified this user; here is proof."

3.4 Authentication vs. Authorization

Before the protocol deep-dive, one distinction must be completely clear, because it is the single most common point of confusion in this entire topic — including among working developers:

Term	Question It Answers	Everyday Analogy
Authentication (AuthN)	Who are you?	Showing your ID card at a building's front desk.
Authorization (AuthZ)	What are you allowed to do?	The visitor badge that only opens certain floors.

COMMON MISTAKE

As you will see below, **OAuth 2.0 is fundamentally an authorization protocol**, not an authentication protocol — even though it is extremely commonly used, in practice, as the foundation for consumer login. Using raw OAuth 2.0 alone to log a user in (without OpenID Connect on top) is a well-known anti-pattern that has caused real security vulnerabilities in production apps.

3.5 SAML 2.0 — Enterprise SSO

Security Assertion Markup Language (SAML) 2.0 is an XML-based standard, published in 2005, that remains the dominant protocol for enterprise SSO — the kind of login a bank employee, a college administrator, or a corporate staff member experiences when a single corporate login unlocks email, an HR system, and a dozen other internal tools. The core artifact SAML passes around is called an **Assertion** — a signed XML document stating who the user is and what attributes (department, role, email) are true about them.

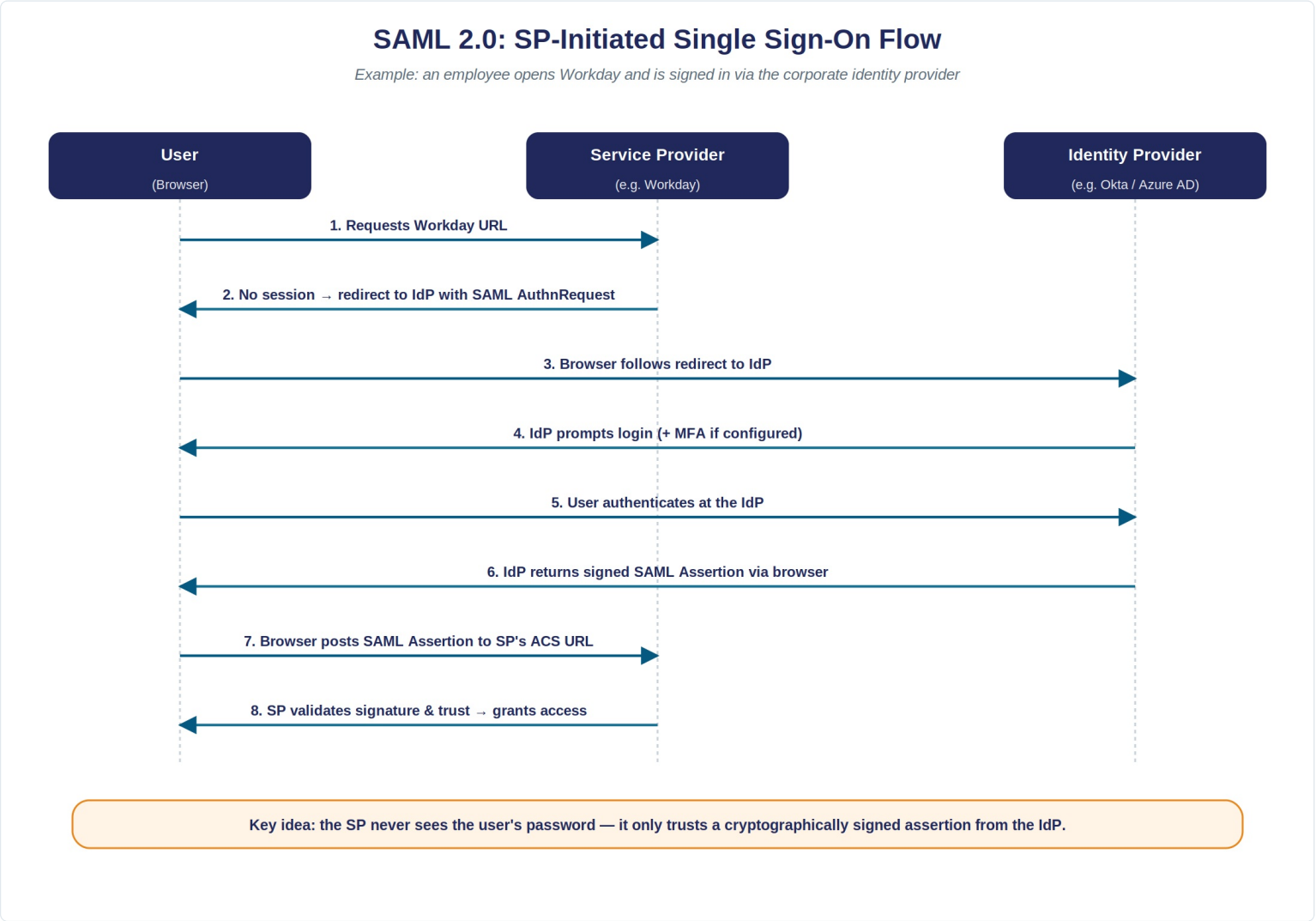


Figure 3.2 — The SP-initiated SAML flow: an employee opens an app, is bounced to the IdP to authenticate, and returns with a signed assertion the app can trust.

EXAMPLE — ENTERPRISE SAML IN PRACTICE

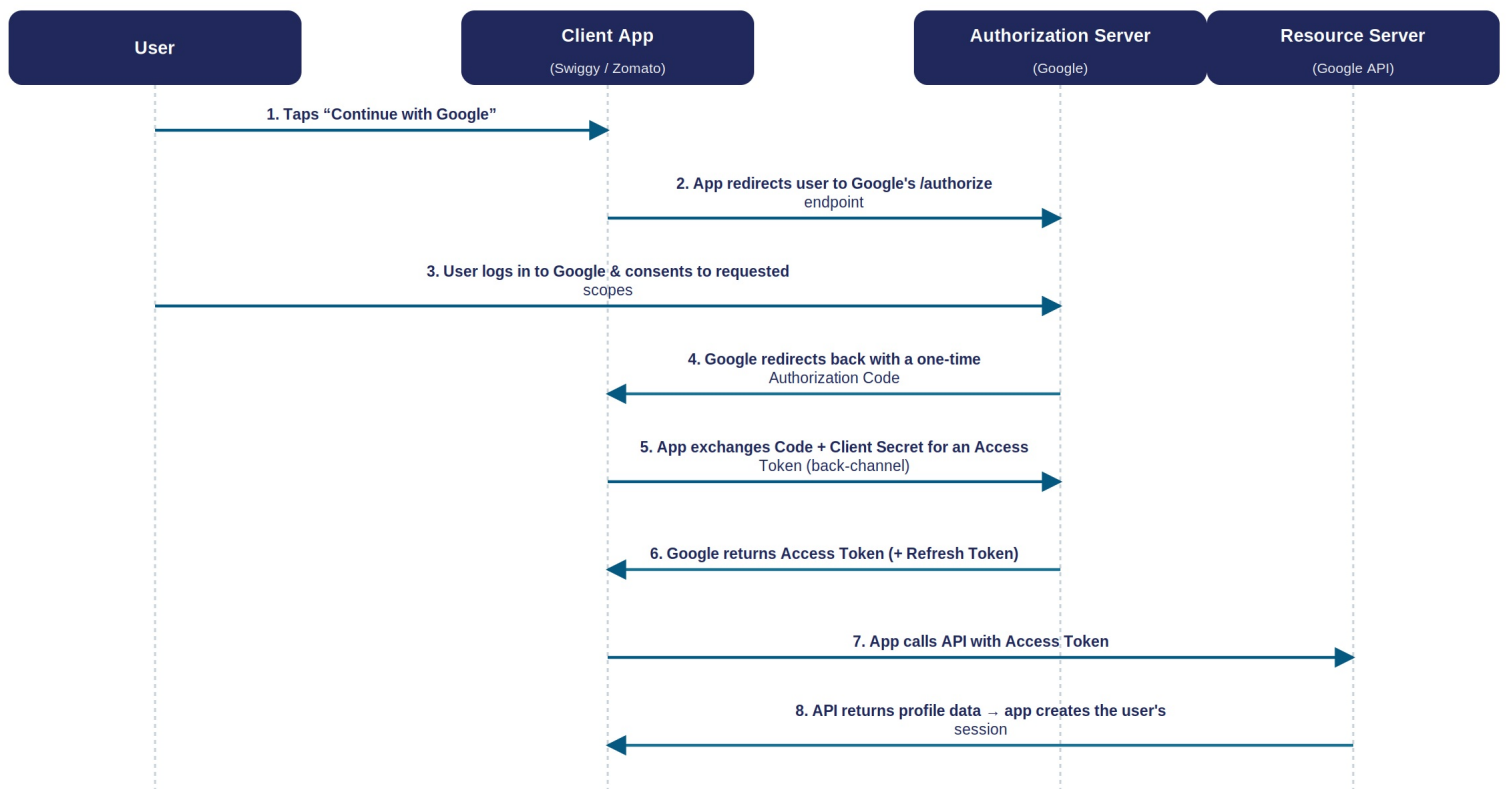
A bank's IT department configures its corporate Identity Provider (commonly Okta or Microsoft Entra ID) as the single source of truth for employee identity. When an employee opens Salesforce or Workday, they are redirected to the bank's own login page (which enforces the bank's own password policy and mandatory MFA), and only after succeeding there do they land back inside Salesforce, already signed in. If the employee leaves the company, disabling their one corporate IdP account instantly revokes their access to every connected application — a critical control for compliance and offboarding.

3.6 OAuth 2.0 — Delegated Authorization

OAuth 2.0 was designed to solve a different problem than SAML: how can a user grant one application *limited* access to their data on another service, **without handing over their password**? The classic motivating example: letting a photo-printing website access only your Google Photos library, without ever giving that website your Google password (which would also let it read your Gmail).

OAuth 2.0: Authorization Code Flow

Example: "Login with Google" on a food-delivery app like Swiggy or Zomato



Key idea: OAuth 2.0 is about AUTHORIZATION — granting the app limited access to your Google data — not proof of identity. For login/identity, OpenID Connect adds an ID Token on top of this same flow.

Figure 3.3 — The OAuth 2.0 Authorization Code flow, the most common and most secure OAuth flow, shown for a "Login with Google" button on a consumer app.

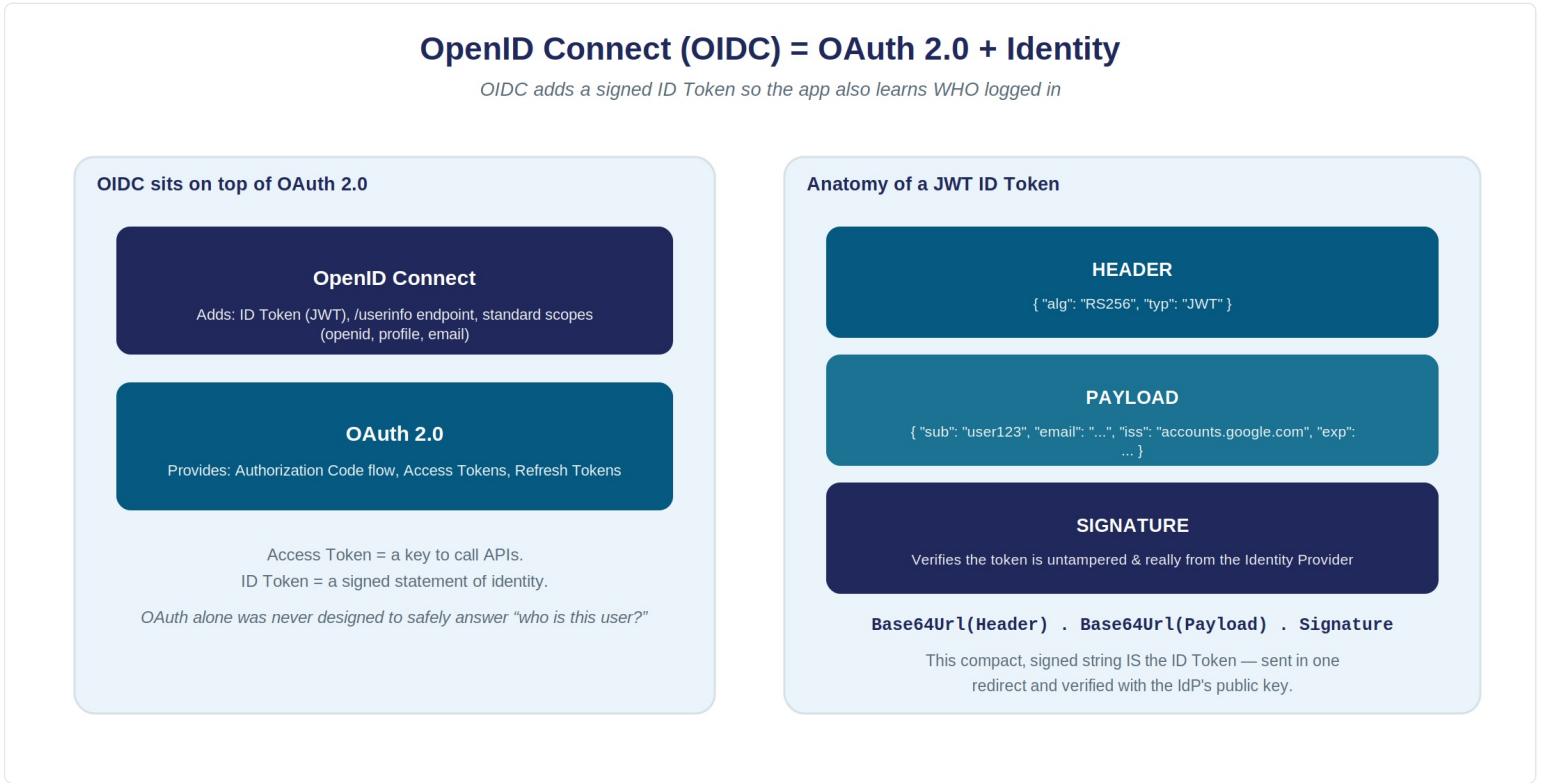
Walking through the eight steps in Figure 3.3: the user taps a "Continue with Google" button inside the app (1); the app redirects the browser to Google's authorization endpoint (2); the user authenticates with Google and approves the specific permissions ("scopes") requested (3); Google redirects back to the app with a short-lived, single-use **authorization code** (4); critically, the app then exchanges that code for an actual **access token** over a secure, server-to-server "back-channel" request that also includes the app's own secret credential (5); Google returns the access token, often along with a longer-lived **refresh token** (6); the app uses the access token to call Google's API (7); and Google returns the requested data, which the app uses to create the user's session (8).

WHY THE BACK-CHANNEL STEP MATTERS

Step 5 happening server-to-server, rather than through the user's browser, is what keeps the app's client secret confidential and prevents the authorization code from being intercepted and reused by an attacker. Mobile and single-page apps, which cannot safely store a secret, use an enhancement called **PKCE (Proof Key for Code Exchange)** to achieve the same protection without needing a stored secret at all.

3.7 OpenID Connect & the JSON Web Token

If OAuth 2.0 is about authorization, how does "Login with Google" actually work as a *login*? The answer is **OpenID Connect (OIDC)**, a thin, standardized identity layer built directly on top of OAuth 2.0. OIDC adds one crucial new artifact to the flow already described above: a signed **ID Token**, formatted as a **JSON Web Token (JWT)**, which makes a verifiable statement about who the user is — not just what data the app may access.



SAML vs. OAuth 2.0 vs. OpenID Connect

Three protocols, three different jobs — choosing the right one matters

SAML 2.0	OAuth 2.0	OpenID Connect
PRIMARY PURPOSE Authentication (enterprise SSO)	PRIMARY PURPOSE Authorization (delegated access)	PRIMARY PURPOSE Authentication + Authorization
DATA FORMAT XML Assertions	DATA FORMAT JSON + Access Tokens	DATA FORMAT JSON + JWT ID Token
TYPICAL USER Employees — enterprise apps	TYPICAL USER Consumers & third-party apps	TYPICAL USER Consumers & mobile apps
COMMON EXAMPLE Login to Workday / Salesforce via corporate IdP	COMMON EXAMPLE "Allow Swiggy to access your Google contacts"	COMMON EXAMPLE "Sign in with Google" on Flipkart / Lenskart
TOKEN CARRIED VIA Browser redirect / POST	TOKEN CARRIED VIA HTTPS back-channel	TOKEN CARRIED VIA HTTPS back-channel
BEST FIT Legacy & enterprise B2B SSO	BEST FIT API access delegation	BEST FIT Modern consumer login / SSO

Figure 3.5 — SAML, OAuth 2.0, and OpenID Connect compared side by side.

RULE OF THUMB

If you are building enterprise workforce SSO into business software, reach for **SAML** (or OIDC, which is increasingly replacing SAML even in enterprise contexts). If you only need to call another service's API on a user's behalf, use **OAuth 2.0**. If you need to know *who* the user is — i.e. you are building a "Sign in with..." button — use **OpenID Connect**.

3.9 Case Study: SSO in Banking & Net-Banking Ecosystems

CASE STUDY

A typical Indian bank's digital ecosystem includes a net-banking web portal, a mobile banking app, and integration with UPI apps for payments — plus, for staff, an entirely separate internal SSO layer (Section 3.5) covering email, HR, and internal tools. For customer-facing SSO, banks operate under significantly tighter constraints than a consumer app: RBI guidelines mandate strong multi-factor authentication for financial transactions, and session tokens are typically extremely short-lived compared to a shopping app.

A customer's identity is frequently anchored to their registered mobile number and a device-level trust signal, with UPI PIN or biometric verification acting as an additional, transaction-level authentication step layered *on top of* the initial SSO-style login — a good real-world illustration that SSO establishes *who you are* once, while sensitive individual actions can still demand additional, targeted re-verification (this is called "step-up authentication").

3.10 Case Study: Lenskart — Social Login & OTP-Based SSO

CASE STUDY

Consumer platforms such as **Lenskart** typically offer two low-friction entry points on their login screen: "Continue with Google" (an OIDC flow, exactly as described in Sections 3.6 and 3.7) and a phone-number-plus-OTP flow, where the platform's own backend acts as the Identity Provider, texting a one-time code to verify the user controls that phone number.

Both are, conceptually, SSO: they let the user skip creating and remembering a Lenskart-specific password by trusting a separate, already-established identity signal — Google's account, or control of a phone number — instead. From a security design standpoint, the OTP flow requires its own careful engineering: rate-limiting OTP requests (to prevent SMS-bombing abuse), short OTP expiry windows, and binding the OTP verification to the specific device/session that requested it, so a leaked OTP alone cannot be replayed from an attacker's device.

3.11 SSO Security Risks & Mitigations

Centralizing authentication brings real benefits, but it also concentrates risk: if the single Identity Provider is compromised, every connected application is potentially compromised too. The table below covers the risks most relevant to a working understanding of production SSO systems.

Risk	What It Is	Mitigation
Token theft / replay	An attacker steals a valid session or access token (e.g. via malware, a compromised network, or an XSS vulnerability) and reuses it to impersonate the user.	Short token lifetimes, token binding to device/IP where feasible, secure (HttpOnly, encrypted) storage of tokens.
SAML XML signature wrapping	A sophisticated attack that manipulates the structure of a signed SAML XML message so a signature that was valid for one part of the document appears to validate a maliciously altered part.	Using well-maintained, standards-compliant SAML libraries rather than custom XML parsing; strict schema validation.
Open redirect	A poorly validated redirect parameter in the login flow is abused to send the user's browser (and sometimes their auth code) to an attacker-controlled site after login.	Strict allow-lists of valid redirect URIs, registered in advance with the Identity Provider.
Phishing via consent screens	A malicious app registers with an IdP and tricks users into approving broad OAuth scopes, gaining legitimate-looking access to their data.	IdP-side app vetting/verification programs, users reviewing requested scopes before approving, minimal scope requests by legitimate apps.
Session fixation	An attacker forces a victim's session to use a session identifier the attacker already knows, then hijacks the session once the victim authenticates.	Always issuing a brand-new session identifier immediately after successful authentication.

3.12 SSO + MFA: Stronger Together

SSO and Multi-Factor Authentication (MFA) solve different problems and work best combined. SSO reduces *how many* passwords exist to attack; MFA makes each individual authentication event harder to defeat even if a password (or the first factor) is compromised, by requiring a second, independent proof — something the user has (a phone, a hardware key) or something the user is (a fingerprint, face recognition).

KEY POINT

Because SSO centralizes authentication into one Identity Provider, that IdP becomes the single highest-value target in the entire system. This is precisely why enforcing MFA *at the IdP* is one of the single highest-leverage security controls an organization can implement — it protects every connected application at once. Many modern IdPs also support **adaptive** or **risk-based authentication**, which only prompts for a second factor when a login looks unusual (new device, new country, impossible travel time since the last login).

3.13 Enterprise SSO Tools Landscape

Provider	Notes
Okta	Independent, widely used enterprise identity platform; strong SAML and OIDC support.
Microsoft Entra ID (formerly Azure AD)	Deeply integrated with Microsoft 365 and Windows environments; very common in large enterprises.
Ping Identity, OneLogin	Independent enterprise identity platforms, similar positioning to Okta.
AWS IAM Identity Center	AWS-native SSO for federating access across many AWS accounts.
Google Workspace / Google Identity	Both an enterprise IdP (Workspace) and one of the most common consumer OIDC providers ("Sign in with Google").

3.14 Classroom Exercise 3 — Map the OAuth Login Flow

CLASSROOM EXERCISE • 12-15 MINUTES

Working individually or in pairs, students fill in the blanks below using Figure 3.3 as a reference, then compare answers with a neighbor before the instructor reveals the full flow.

1. The user taps "Continue with Google" inside the _____ (name the role, not the company).
2. The app redirects the browser to Google's _____ endpoint.
3. After the user logs in and consents, Google returns a short-lived, single-use _____.
4. The app exchanges that value for an access token using a _____-channel request (not visible to the browser).
5. Which single artifact, added by OpenID Connect, would let the app know the user's identity, not just their permission to call an API?

3.15 Classroom Exercise 4 — Rapid-Fire Protocol Quiz

CLASSROOM EXERCISE • TRUE / FALSE • 5-8 MINUTES

1. SAML is primarily an authorization protocol. (False — authentication)
2. OAuth 2.0 alone is sufficient, and safe, for logging a user into an app. (False — use OIDC)
3. A JWT's signature can be verified without calling the Identity Provider back. (True)
4. In SSO, the Service Provider directly checks the user's password. (False — the IdP does)
5. MFA and SSO solve the same problem and are redundant together. (False — complementary)
6. PKCE exists because mobile/single-page apps cannot safely store a client secret. (True)

3.16 Module 3 Summary & Key Takeaways

KEY TAKEAWAYS

- SSO lets a user authenticate once with a trusted Identity Provider and access many Service Providers, without those services ever seeing the user's password.
- Authentication (who are you) and authorization (what can you do) are different questions — and conflating them is the single most common mistake in this space.
- SAML (XML-based) dominates classic enterprise SSO; OAuth 2.0 handles delegated API authorization; OpenID Connect adds identity on top of OAuth 2.0 via a signed JWT ID Token, and now dominates consumer "Sign in with..." experiences.
- Centralizing identity concentrates risk onto the Identity Provider — making MFA at the IdP one of the highest-leverage controls available to any organization.
- Real systems — from food-delivery apps to banks — combine SSO for initial login with step-up authentication (OTP, biometrics) for sensitive individual actions.

Recap, Glossary & References

Recap: How the Three Topics Fit Together

Return, one last time, to the case-study architecture from Module 1 (Figure 1.3). A request from a user's phone first meets the **Login / SSO layer** (Module 3) — OAuth 2.0 and OpenID Connect verifying who the user is, perhaps backed by a social IdP or the platform's own OTP flow. Once inside, every layer of the system behind that front door — the API gateway, the microservices, the storage, the database — must be continuously verified as correctly configured, which is the job of **CSPM** (Module 2), operating against the backdrop of the **Shared Responsibility Model** (Module 1) that defines exactly which of those layers are even the platform's job to secure in the first place. Identity at the front door, and posture behind it, are the two questions a cloud security program must always be able to answer.

Interview & Exam Pointers

- Be ready to **draw** the shared responsibility model from memory and correctly place at least five example controls on it.
- Be able to explain, in one sentence each, the difference between CSPM, CWPP, and CIEM — this is a very common cloud-security interview question.
- Be able to whiteboard the OAuth 2.0 Authorization Code flow, labeling which step is the "back-channel" step and explaining why it matters.
- Be ready to explain, precisely, why OAuth 2.0 alone is not an authentication protocol.
- Have one real or realistic breach example ready to discuss in depth (Capital One is the standard reference for shared-responsibility failures).

Glossary of Key Terms

Access Token

A credential (often short-lived) that grants the bearer permission to call a specific API on a user's behalf.

Assertion (SAML)

A signed XML statement issued by an IdP, asserting facts about an authenticated user.

CIEM

Cloud Infrastructure Entitlement Management — finds and reduces over-privileged cloud identities.

CNAPP

Cloud-Native Application Protection Platform — a unified platform bundling CSPM, CWPP, CIEM, and related capabilities.

CSPM

Cloud Security Posture Management — continuous, automated discovery and remediation of cloud misconfigurations.

CWPP

Cloud Workload Protection Platform — runtime security for VMs, containers, and serverless functions.

Identity Provider (IdP)

The system that authenticates a user and issues a signed proof of identity to other applications.

JWT (JSON Web Token)

A compact, signed, three-part token format (header.payload.signature) commonly used to carry OIDC ID Tokens.

KSPM

Kubernetes Security Posture Management — CSPM principles applied to clusters, pods, and RBAC.

Least Privilege

The principle of granting an identity only the minimum access it needs to do its job, and no more.

MFA (Multi-Factor Authentication)

Requiring two or more independent proofs of identity before granting access.

OAuth 2.0

A delegated-authorization protocol allowing an app limited access to a user's data on another service, without seeing their password.

OpenID Connect (OIDC)

An identity layer built on top of OAuth 2.0, adding a signed ID Token that proves who the user is.

SAML 2.0

An XML-based standard for exchanging authentication and authorization data, dominant in enterprise SSO.

Service Provider (SP) / Relying Party

The application a user wants to access, which trusts an IdP's assertion instead of checking credentials itself.

Shared Responsibility Model

The framework defining which security controls the cloud provider handles versus the customer, varying by IaaS/PaaS/SaaS.

SSO (Single Sign-On)

Logging in once to gain access to multiple, independent applications.

Answer Key for Classroom Exercises

Exercise 1 — Whose Responsibility Is It?

1. Cloud Provider (physical hardware).
2. Customer (a configuration choice).
3. Customer (an access-control policy choice, though the IdP's MFA *capability* is provided by the vendor).
4. Customer (data protection is never fully delegated, even on SaaS).
5. Cloud Provider (physical/data-center security).
6. Customer (network configuration inside their own account).
7. Cloud Provider (virtualization layer).

Exercise 2 — Spot the Misconfiguration

All six settings are problems, but the **public access on a bucket literally named "prod-customer-exports"** is the single most urgent — it is an active, ongoing data exposure. Enabling **encryption at rest** would not have prevented discovery, but would have reduced the impact if the data were still somehow accessed. The **"AdministratorAccess" role attached** is a distinct least-privilege violation: even resources with no data-exposure issue should never be granted broader permissions than they need.

Exercise 3 — Map the OAuth Login Flow

(1) Client App / Service Provider (2) authorization (/authorize) (3) Authorization Code (4) back (5) The OpenID Connect ID Token (JWT).

Further Reading & References

- NIST Special Publication 800-145, "The NIST Definition of Cloud Computing."
- Cloud Security Alliance (CSA), "Security Guidance for Critical Areas of Focus in Cloud Computing."
- OASIS, "Security Assertion Markup Language (SAML) V2.0" specification.
- IETF RFC 6749, "The OAuth 2.0 Authorization Framework."
- OpenID Foundation, "OpenID Connect Core 1.0" specification.
- CIS (Center for Internet Security) Benchmarks for AWS, Azure, and GCP.
- Reserve Bank of India, Cyber Security Framework for Banks / NBFCs.
- Publicly reported post-incident analyses of the 2019 Capital One breach (U.S. Senate and regulatory filings).

END OF COURSE MATERIAL

Thank you for teaching — or studying — this material. The accompanying slide deck mirrors this document's three-module structure and includes full speaker notes for classroom delivery.